VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Practical No. 02

Aim: Write a program to perform Bubble sort and Selection Sort for any given list of numbers.

Software Requirement: Python IDE, PyCharm, Visual Studio Code (VS Code), Jupyter Notebook.

Theory:

Bubble Sort:

Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. It is named "Bubble Sort" because smaller elements "bubble" to the top of the list. While not the most efficient sorting algorithm, it is easy to understand and implement, making it a good starting point for learning about sorting algorithms. In this article, we will delve into the concept of Bubble Sort, provide a Python implementation with output, and discuss the time complexity of the algorithm.

Bubble Sort algorithm:

1. Iterate through the list: Start from the beginning of the list and compare each pair of adjacent elements.
2. Compare and swap: If the elements are in the wrong order, swap them. This ensures that the larger element "bubbles up" and the smaller one moves down.
3. Continue iterating: Repeat the process for each pair of adjacent elements until the end of the list is reached.
4. Repeat until sorted: Keep iterating through the list until no more swaps are needed. At this point, the list is sorted.

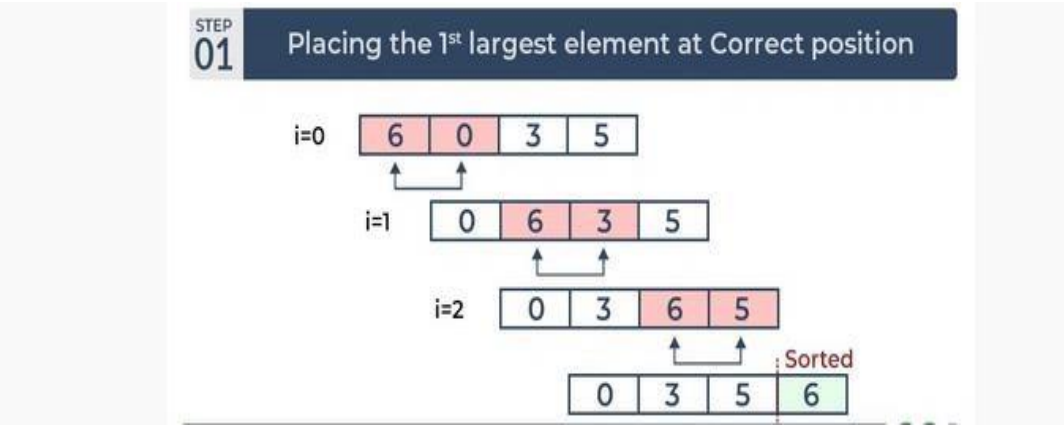
Example:

Input: arr[] = {6, 0, 3, 5}

First Pass:

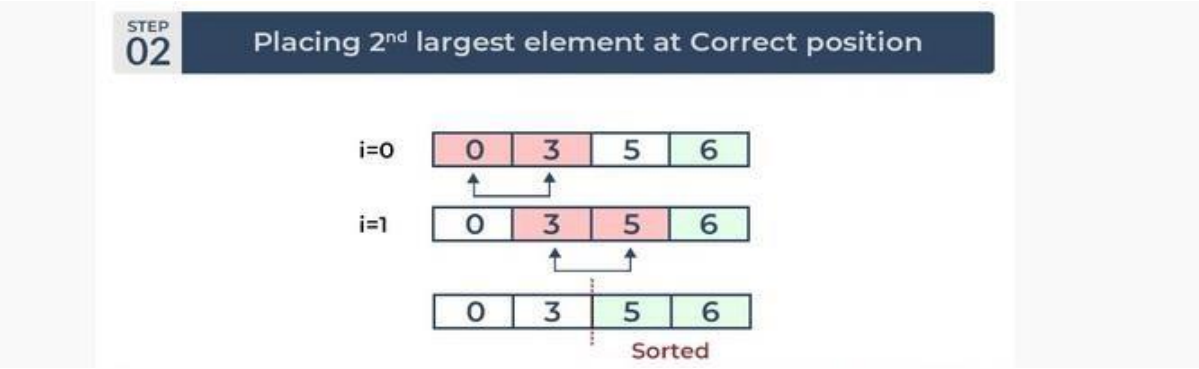
The largest element is placed in its correct position, i.e., the end of the array.

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.



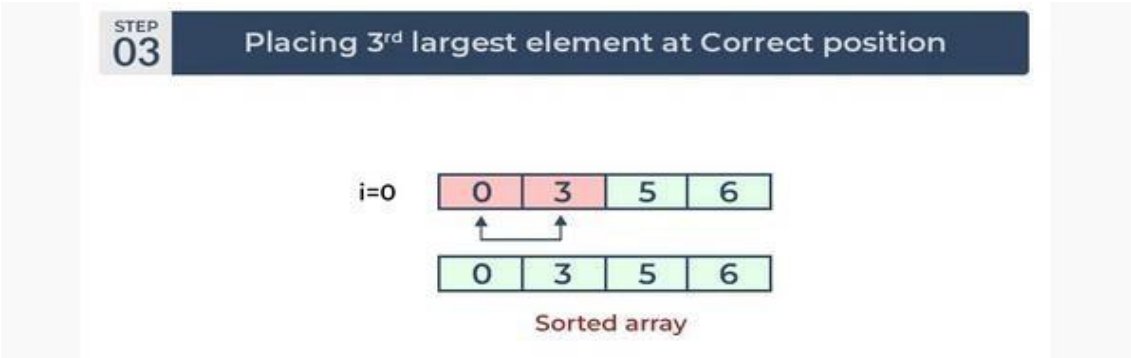
Second Pass:

Place the second largest element at correct position



Third Pass:

Place the remaining two elements at their correct positions.



- Total no. of passes: n-1
- Total no. of comparisons: n*(n-1)/2

Advantages of Bubble Sort:

- Bubble sort is easy to understand and implement.
- It does not require any additional memory space.
- It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.

Disadvantages of Bubble Sort:

- Bubble sort has a time complexity of $O(N^2)$ which makes it very slow for large data sets.
- Bubble sort is a comparison-based sorting algorithm, which means that it requires a comparison operator to determine the relative order of elements in the input data set. It can limit the efficiency of the algorithm in certain cases.

Selection Sort:

Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.

Selection sort Algorithm:

The following are the steps to explain the working of the Selection sort in Python.

Step - 1: Get the length of the array.
length = len(array) → 6

Step - 2: First, we set the first element as minimum element.

Step - 3: Now compare the minimum with the second element. If the second element is smaller than the first, we assign it as a minimum.

Again we compare the second element to the third and if the third element is smaller than second, assign it as minimum. This process goes on until we find the last element.

Step - 4: After each iteration, minimum element is swapped in front of the unsorted array.

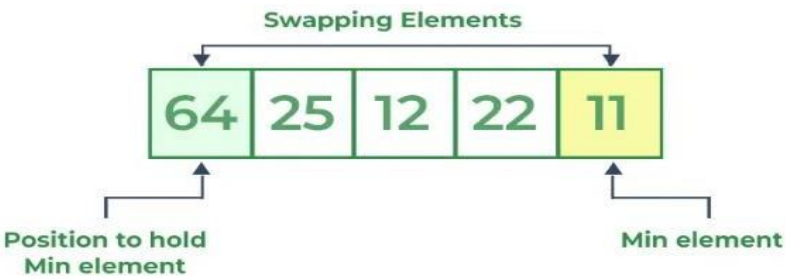
Step - 5: The second to third steps are repeated until we get the sorted array.

Example: Let's consider the following array as an example: arr[] = {64, 25, 12, 22, 11}

First pass:

• For the first position in the sorted array, the whole array is traversed from index 0 to 4 sequentially. The first position where 64 is stored presently, after traversing whole array it is clear that 11 is the lowest value.

• Thus, replace 64 with 11. After one iteration 11, which happens to be the least value in the array, tends to appear in the first position of the sorted list.



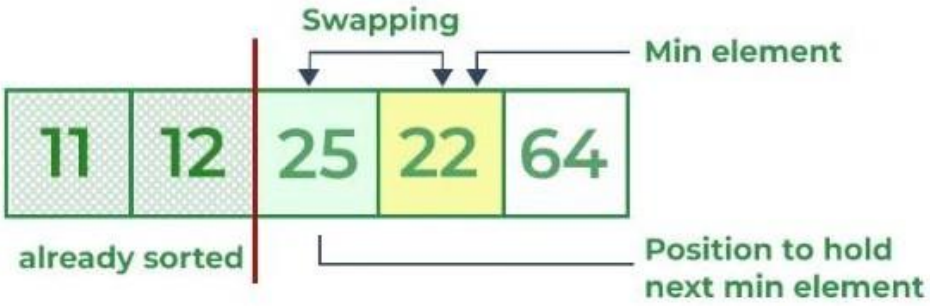
Second Pass:

- For the second position, where 25 is present, again traverse the rest of the array in a sequential manner.
- After traversing, we found that **12** is the second lowest value in the array and it should appear at the second place in the array, thus swap these values.

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Third Pass:

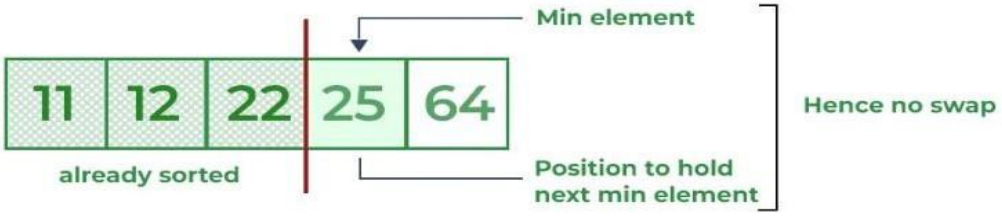
- Now, for third place, where **25** is present again traverse the rest of the array and find the third least value present in the array.
- While traversing, **22** came out to be the third least value and it should appear at the third place in the array, thus swap **22** with element present at third position.



VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Fourth pass:

- Similarly, for fourth position traverse the rest of the array and find the fourth least element in the array.
- As **25** is the 4th lowest value hence, it will place at the fourth position.



Fifth Pass:

- At last the largest value present in the array automatically get placed at the last position in the array
- The resulted array is the sorted array.

<u>Advantages of Selection Sort Algorithm</u>

- Simple and easy to understand.
- Works well with small datasets.

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Disadvantages of the Selection Sort Algorithm

- Selection sort has a time complexity of O(n^2) in the worst and average case.
- Does not work well on large datasets.
- Does not preserve the relative order of items with equal keys which means it is not stable.

Program:

For Bubble Sort

```
list1 = [9, 16, 6, 26, 0]
print("Unsorted list1 is", list1)
# Bubble sort
for j in range(len(list1) - 1):  # Outer loop
    for i in range(len(list1) - 1):  # Inner loop
        if list1[i] > list1[i + 1]:  # Compare adjacent elements
            list1[i], list1[i + 1] = list1[i + 1], list1[i]
    print("List after iteration", j + 1, "is", list1)  # Print list after each outer loop iteration

print("\nSorted list is", list1)
```

Output:

```
Unsorted list1 is [9, 16, 6, 26, 0]
List after iteration 1 is [9, 6, 16, 0, 26]
List after iteration 2 is [6, 9, 0, 16, 26]
List after iteration 3 is [6, 0, 9, 16, 26]
List after iteration 4 is [0, 6, 9, 16, 26]

Sorted list is [0, 6, 9, 16, 26]
```

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

For Selection Sort

```
def selection_sort(arr):
    # Traverse through all array elements
    for i in range(len(arr)):
        min_index = i  # Assume the current index as the minimum
        # Find the minimum element in the remaining unsorted array
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_index]:
                min_index = j  # Update the minimum index if a smaller element is found
        # Swap the found minimum element with the first element of the unsorted part
        arr[i], arr[min_index] = arr[min_index], arr[i]

# Example usage
arr = [64, 25, 12, 22, 11]
print("Original array:", arr)
selection_sort(arr)
print("Sorted array:", arr)
```

Output:

```
Original array: [64, 25, 12, 22, 11]
Sorted array: [11, 12, 22, 25, 64]
```

Conclusion:

Thus, I have executed the Python program to perform Bubble Sort and Selection Sort Successfully.

JAIDEV EDUCATION SOCIETY'S

J D COLLEGE OF ENGINEERING AND MANAGEMENT

KATOL ROAD, NAGPUR

Website: www.jdcoem.ac.in **E-mail:** info@jdcoem.ac.in

(An Autonomous Institute, with NAAC "A" Grade)

Affiliated to DBATU, RTMNU & MSBTE Mumbai

Department of Artificial Intelligence

"A Place to Learn, A Chance to Grow"

Session: 2024-25

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.